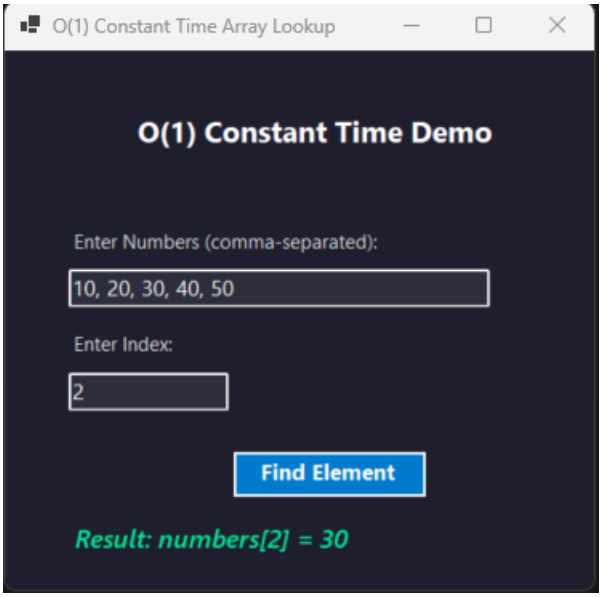
O(1) — Constant Time Ex. 1



Code:

```
using System;

using System.Drawing;

using System.Linq;

using System.Windows.Forms;


namespace O_1____Constant_Time_Ex._1

{

  public partial class Form1 : Form

  {

    public Form1()

    {

      InitializeComponent();

    }


    private void btnGetElement_Click(object sender, EventArgs e)

    {
```

```
// 1. Get array values entered by the user
string rawNumbers = txtArrayInput.Text.Trim();

if (string.IsNullOrEmpty(rawNumbers))
{
   ShowError("Please enter numbers separated by commas.");
   return;
}

int[] numbers;
try
{
   numbers = rawNumbers
      .Split(new[] { ',', ' ' }, StringSplitOptions.RemoveEmptyEntries)
      .Select(int.Parse)
      .ToArray();
}
catch (FormatException)
{
   ShowError("Invalid input! Use numbers separated by commas (e.g., 10, 20, 30).");
   return;
}

// 2. Get index entered by the user
if (!int.TryParse(txtIndex.Text.Trim(), out int index))
{
```

```
            ShowError("Please enter a valid whole number for the index.");
            return;
        }

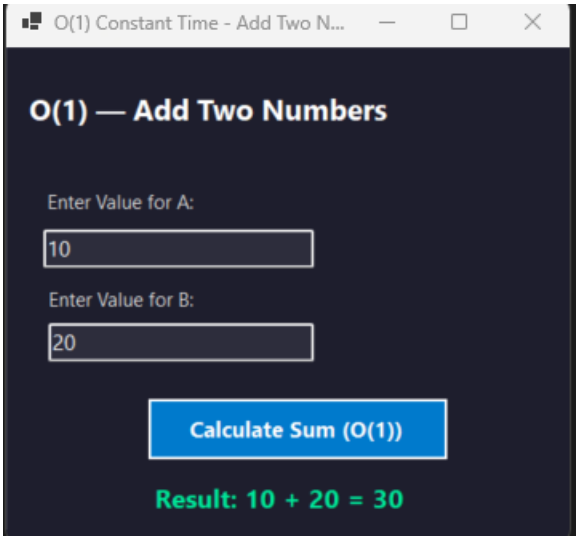
        // 3. Prevent IndexOutOfRangeException
        if (index < 0 || index >= numbers.Length)
        {
            ShowError($"Index out of bounds! Valid index range is 0 to {numbers.Length - 1}.");
            return;
        }

        // 4. O(1) Constant Time Array Lookup
        int answer = numbers[index];

        // 5. Output answer
        lblResult.ForeColor = Color.FromArgb(0, 230, 153);
        lblResult.Text = $"Result: numbers[{index}] = {answer}";
    }

    private void ShowError(string message)
    {
        lblResult.ForeColor = Color.FromArgb(255, 100, 100);
        lblResult.Text = message;
    }
  }
}
```

O(1) — Constant Time Ex. 2



Code:

```
using System;

using System.Drawing;

using System.Windows.Forms;


namespace O_1____Constant_Time_Ex._2

{

  public partial class Form1 : Form

  {

    public Form1()

    {

      InitializeComponent();

    }


    private void btnAdd_Click(object sender, EventArgs e)

    {
```

```
    // Parse Number A
    if (!int.TryParse(txtNumA.Text.Trim(), out int a))
    {
        ShowError("Please enter a valid integer for A.");
        return;
    }

    // Parse Number B
    if (!int.TryParse(txtNumB.Text.Trim(), out int b))
    {
        ShowError("Please enter a valid integer for B.");
        return;
    }

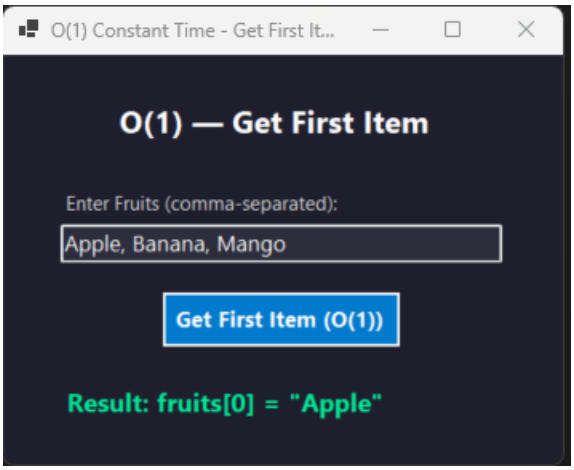
    // O(1) Constant Time Addition: a + b
    int sum = a + b;

    // Display success output
    lblResult.ForeColor = Color.FromArgb(0, 230, 153); // Accent Green
    lblResult.Text = $"Result: {a} + {b} = {sum}";
}

private void ShowError(string message)
{
    lblResult.ForeColor = Color.FromArgb(255, 100, 100); // Soft Red
    lblResult.Text = message;
```

```
    }
  }
}
```

O(1) — Constant Time Ex. 3



Code:

```
using System;
using System.Drawing;
using System.Linq;
using System.Windows.Forms;

namespace O_1____Constant_Time_Ex._3
{
  public partial class Form1 : Form
  {
    public Form1()
    {
      InitializeComponent();
    }
```

```
private void btnGetFirst_Click(object sender, EventArgs e)
{
    string rawText = txtFruitsInput.Text.Trim();

    if (string.IsNullOrEmpty(rawText))
    {
        ShowError("Please enter at least one item.");
        return;
    }

    string[] fruits = rawText
        .Split(new[] { ',' }, StringSplitOptions.RemoveEmptyEntries)
        .Select(item => item.Trim())
        .ToArray();

    if (fruits.Length == 0)
    {
        ShowError("No valid items found.");
        return;
    }

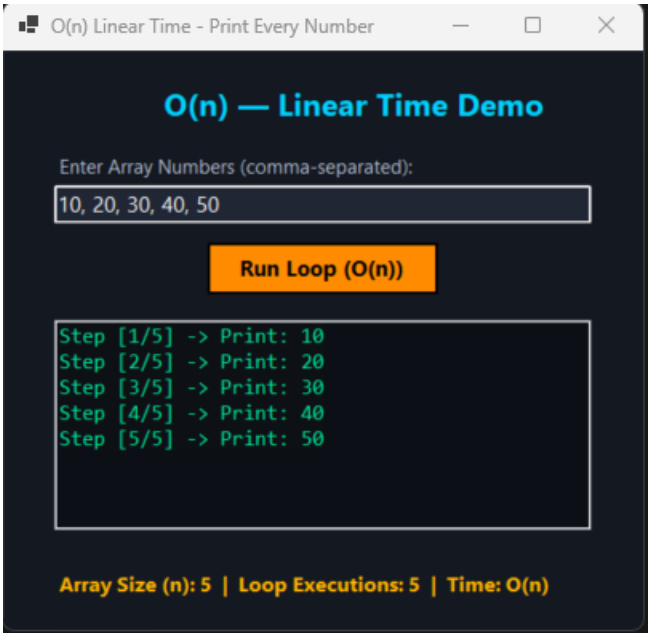
    // O(1) Constant Time Lookup
    string firstFruit = fruits[0];

    lblResult.ForeColor = Color.FromArgb(0, 230, 153);
```

```
        lblResult.Text = $"Result: fruits[0] = \"{firstFruit}\"";
    }

    private void ShowError(string message)
    {
        lblResult.ForeColor = Color.FromArgb(255, 100, 100);
        lblResult.Text = message;
    }
  }
}
```

O(n) — Linear Time Ex. 1



Code:

```
using System;

using System.Drawing;

using System.Linq;

using System.Windows.Forms;
```

```
namespace O_1____Linear_Time_Ex._1
{
   public partial class Form1 : Form
   {
      public Form1()
      {
         InitializeComponent();
      }

      private void btnPrintAll_Click(object sender, EventArgs e)
      {
         lstOutput.Items.Clear();

         string rawInput = txtNumbersInput.Text.Trim();

         if (string.IsNullOrEmpty(rawInput))
         {
            lstOutput.Items.Add("Error: Input cannot be empty.");
            return;
         }

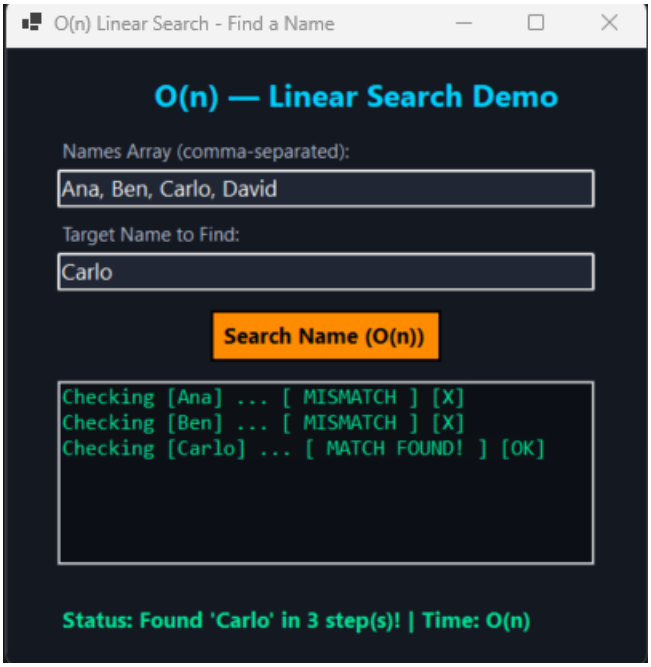
         int[] numbers;
         try
         {
            numbers = rawInput
```

```
            .Split(new[] { ',', ' ' }, StringSplitOptions.RemoveEmptyEntries)
            .Select(int.Parse)
            .ToArray();
        }
        catch (FormatException)
        {
            lstOutput.Items.Add("Error: Please enter valid numbers separated by commas.");
            return;
        }

        // O(n) Linear Time Loop
        for (int i = 0; i < numbers.Length; i++)
        {
            lstOutput.Items.Add($"Step [{i + 1}/{numbers.Length}] -> Print: {numbers[i]}");
        }

        int n = numbers.Length;
        lblStats.ForeColor = Color.FromArgb(255, 180, 0);
        lblStats.Text = $"Array Size (n): {n} | Loop Executions: {n} | Time: O(n)";
    }
  }
}
```

O(n) — Linear Time Ex. 2



Code:

```
using System;

using System.Drawing;

using System.Linq;

using System.Windows.Forms;


namespace O_n____Linear_Time_Ex._2

{

  public partial class Form1 : Form

  {

    public Form1()

    {

      InitializeComponent();

    }
```

```
private void btnSearch_Click(object sender, EventArgs e)
{
    lstSearchTrace.Items.Clear();

    string rawNames = txtNames.Text.Trim();
    string target = txtTarget.Text.Trim();

    if (string.IsNullOrEmpty(rawNames) || string.IsNullOrEmpty(target))
    {
        lblStatus.ForeColor  = Color.FromArgb(255, 100, 100);
        lblStatus.Text  = "Status:  Please enter both array names and a target name.";
        return;
    }

    string[] names = rawNames
        .Split(new[] { ',' }, StringSplitOptions.RemoveEmptyEntries)
        .Select(n => n.Trim())
        .ToArray();

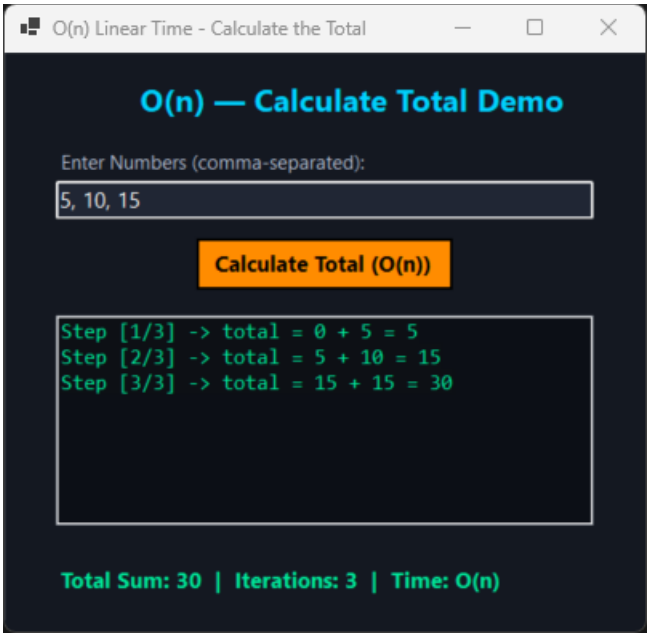
    bool found = false;
    int steps = 0;

    // O(n) Linear Search Loop
    for (int i = 0; i < names.Length; i++)
    {
        steps++;
```

```
            if (string.Equals(names[i], target, StringComparison.OrdinalIgnoreCase))
            {
                lstSearchTrace.Items.Add($"Checking [{names[i]}] ... [ MATCH FOUND! ] [OK]");
                found = true;
                break; // Target found, stop scanning
            }
            else
            {
                lstSearchTrace.Items.Add($"Checking [{names[i]}] ... [ MISMATCH ] [X]");
            }
        }

        if (found)
        {
            lblStatus.ForeColor = Color.FromArgb(0, 230, 153);
            lblStatus.Text = $"Status: Found '{target}' in {steps} step(s)! | Time: O(n)";
        }
        else
        {
            lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
            lblStatus.Text = $"Status: '{target}' not found after {steps} step(s). | Time: O(n)";
        }
    }
  }
}
```

O(n) — Linear Time Ex. 3



Code:

```
using System;

using System.Drawing;

using System.Linq;

using System.Windows.Forms;


namespace O_n____Linear_Time_Ex._3

{

  public partial class Form1 : Form

  {

    public Form1()

    {

      InitializeComponent();

    }


    private void btnCalculate_Click(object sender, EventArgs e)
```

```
{
    lstAccumulationTrace.Items.Clear();

    string rawInput = txtNumbersInput.Text.Trim();

    if (string.IsNullOrEmpty(rawInput))
    {
        lblResult.ForeColor = Color.FromArgb(255, 100, 100);
        lblResult.Text = "Please enter valid numbers.";
        return;
    }

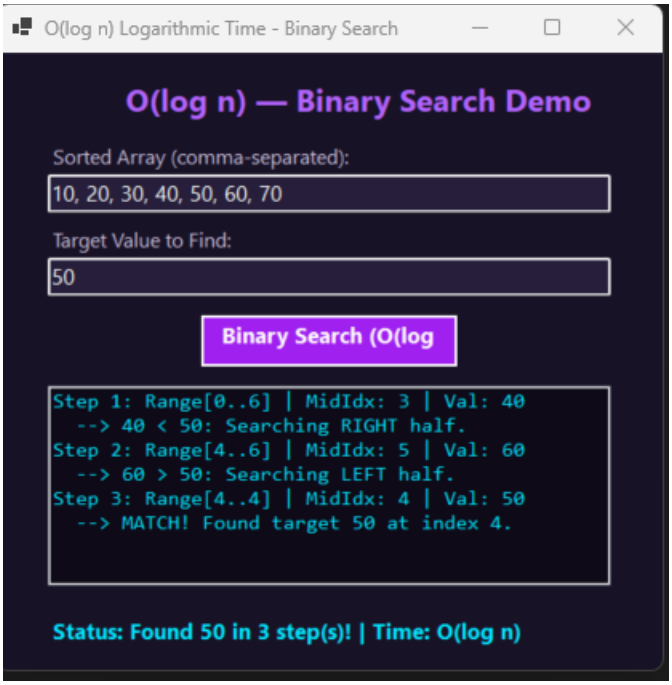
    int[] numbers;
    try
    {
        numbers = rawInput
            .Split(new[] { ',', ' ' }, StringSplitOptions.RemoveEmptyEntries)
            .Select(int.Parse)
            .ToArray();
    }
    catch (FormatException)
    {
        lblResult.ForeColor = Color.FromArgb(255, 100, 100);
        lblResult.Text = "Error: Invalid number format.";
        return;
    }
```

```
        int total = 0;
        int step = 0;

        // O(n) Linear Accumulation Loop
        foreach (int number in numbers)
        {
            step++;
            int previousTotal = total;
            total += number;
            lstAccumulationTrace.Items.Add($"Step [{step}/{numbers.Length}] -> total = {previousTotal} + {number} = {total}");
        }

        lblResult.ForeColor = Color.FromArgb(0, 230, 153);
        lblResult.Text = $"Total Sum: {total} | Iterations: {step} | Time: O(n)";
    }
  }
}
```

O(log n) — Logarithmic Time Ex. 1



Code:

```
using System;

using System.Drawing;

using System.Linq;

using System.Windows.Forms;


namespace O_log_n____Logarithmic_Time_Ex._1

{

  public partial class Form1 : Form

  {

    public Form1()

    {

      InitializeComponent();

    }
```

```
private void btnSearch_Click(object sender, EventArgs e)
{
    lstTrace.Items.Clear();

    string rawArray = txtArrayInput.Text.Trim();
    string rawTarget = txtTarget.Text.Trim();

    if (string.IsNullOrEmpty(rawArray) || string.IsNullOrEmpty(rawTarget))
    {
        lblStatus.ForeColor  = Color.FromArgb(255, 100, 100);
        lblStatus.Text  = "Status:  Please enter both sorted array and target.";
        return;
    }

    int[] numbers;
    int target;

    try
    {
        numbers = rawArray
            .Split(new[] { ',', ' ' }, StringSplitOptions.RemoveEmptyEntries)
            .Select(int.Parse)
            .ToArray();
        target = int.Parse(rawTarget);
    }
    catch (FormatException)
```

```
        {
            lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
            lblStatus.Text = "Status: Invalid input format. Enter integers only.";
            return;
        }

        // O(log n) Binary Search Implementation
        int left = 0;
        int right = numbers.Length - 1;
        int steps = 0;
        bool found = false;

        while (left <= right)
        {
            steps++;
            int middle = (left + right) / 2;
            int currentVal = numbers[middle];

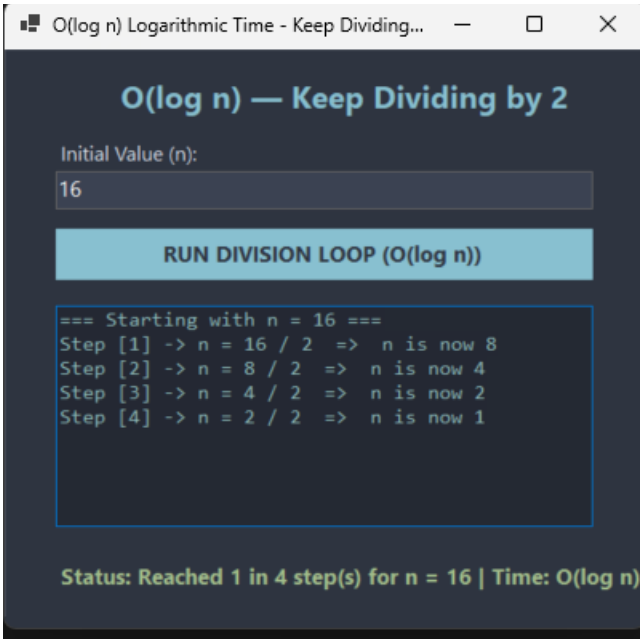
            lstTrace.Items.Add($"Step {steps}: Range[{left}..{right}] | MidIdx: {middle} | Val: {currentVal}");

            if (currentVal == target)
            {
                lstTrace.Items.Add($"  --> MATCH! Found target {target} at index {middle}.");
                found = true;
                break;
```

```
            }
            else if (currentVal < target)
            {
                lstTrace.Items.Add($"  --> {currentVal} < {target}: Searching RIGHT half.");
                left = middle + 1;
            }
            else
            {
                lstTrace.Items.Add($"  --> {currentVal} > {target}: Searching LEFT half.");
                right = middle - 1;
            }
        }

        if (found)
        {
            lblStatus.ForeColor = Color.FromArgb(0, 230, 255);
            lblStatus.Text = $"Status: Found {target} in {steps} step(s)! | Time: O(log n)";
        }
        else
        {
            lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
            lblStatus.Text = $"Status: {target} not found after {steps} step(s). | Time: O(log n)";
        }
    }
  }
}
```

O(log n) — Logarithmic Time Ex. 2



Code:

```
using System;

using System.Drawing;

using System.Windows.Forms;


namespace O_log_n____Logarithmic_Time_Ex._2

{

  public partial class Form1 : Form

  {

    public Form1()

    {

      InitializeComponent();

    }


    private void btnRun_Click(object sender, EventArgs e)

    {
```

```
lstTrace.Items.Clear();

string rawInput = txtNumberInput.Text.Trim();

if (string.IsNullOrEmpty(rawInput) || !int.TryParse(rawInput, out int n) || n <= 1)
{
   lblStatus.ForeColor = Color.FromArgb(191, 97, 106); // Frost Red
   lblStatus.Text = "Status: Enter an integer greater than 1.";
   return;
}

int originalN = n;
int steps = 0;

lstTrace.Items.Add($"=== Starting with n = {n} ===");

// O(log n) Division Loop
while (n > 1)
{
   steps++;
   int previousN = n;
   n = n / 2;
   lstTrace.Items.Add($"Step [{steps}] -> n = {previousN} / 2  =>  n is now {n}");
}

lblStatus.ForeColor = Color.FromArgb(163, 190, 140); // Frost Green
```

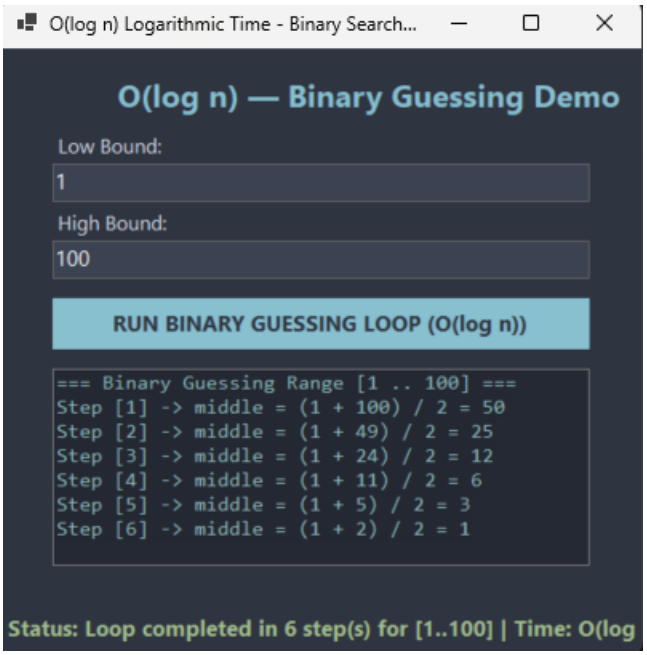
lblStatus.Text = $"Status: Reached 1 in {steps} step(s) for n = {originalN} | Time: O(log n)";

}

}

}

O(log n) — Logarithmic Time Ex. 3



Code:

```
using System;

using System.Drawing;

using System.Windows.Forms;


namespace O_log_n____Logarithmic_Time_Ex._2

{

  public partial class Form1 : Form

  {

    public Form1()
```

```
{
    InitializeComponent();
}

private void btnRun_Click(object sender, EventArgs e)
{
    lstTrace.Items.Clear();

    string rawInput = txtNumberInput.Text.Trim();

    if (string.IsNullOrEmpty(rawInput) || !int.TryParse(rawInput, out int n) || n <= 1)
    {
        lblStatus.ForeColor  = Color.FromArgb(191, 97, 106); // Frost Red
        lblStatus.Text  = "Status: Enter an integer greater than 1.";
        return;
    }

    int originalN = n;
    int steps = 0;

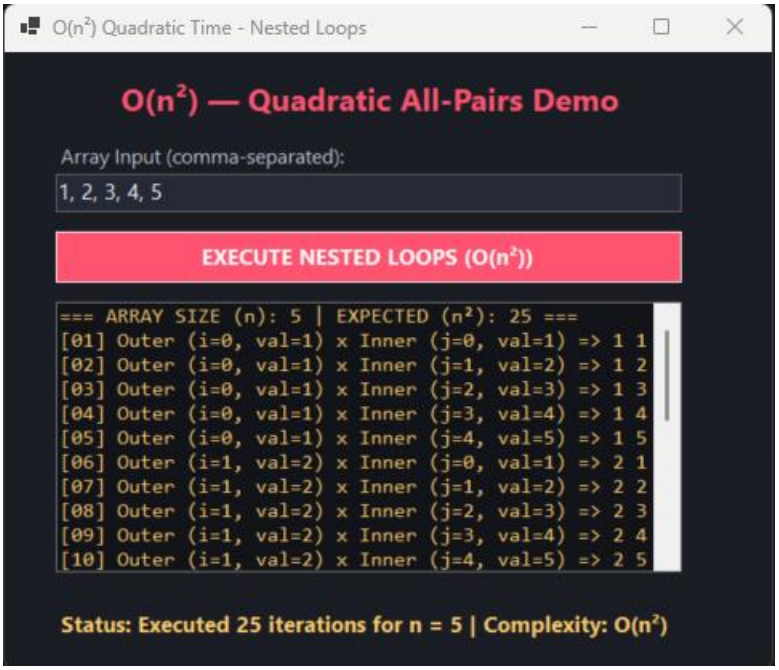
    lstTrace.Items.Add($"=== Starting with n = {n} ===");

    // O(log n) Division Loop
    while (n > 1)
    {
        steps++;
```

```
            int previousN = n;
            n = n / 2;
            lstTrace.Items.Add($"Step [{steps}] -> n = {previousN} / 2  =>  n is now {n}");
        }

        lblStatus.ForeColor = Color.FromArgb(163, 190, 140); // Frost Green
        lblStatus.Text = $"Status: Reached 1 in {steps} step(s) for n = {originalN} | Time: O(log n)";
    }
  }
}
```

O(n^2) — Quadratic Time Ex. 1



Code:

```
using System;
using System.Drawing;
```

```
using System.Linq;
using System.Windows.Forms;

namespace O_n_____Quadratic_Time_Ex._1
{
   public partial class Form1 : Form
   {
      public Form1()
      {
         InitializeComponent();
      }

      private void btnRun_Click(object sender, EventArgs e)
      {
         lstTrace.Items.Clear();

         string rawArray = txtArrayInput.Text.Trim();

         if (string.IsNullOrEmpty(rawArray))
         {
            lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
            lblStatus.Text = "Status: Please enter array numbers.";
            return;
         }

         int[] numbers;
```

```
try
{
    numbers = rawArray
        .Split(new[] { ',', ' ' }, StringSplitOptions.RemoveEmptyEntries)
        .Select(int.Parse)
        .ToArray();
}
catch (FormatException)
{
    lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
    lblStatus.Text = "Status: Invalid input format. Enter integers only.";
    return;
}

int n = numbers.Length;
int totalIterations = 0;

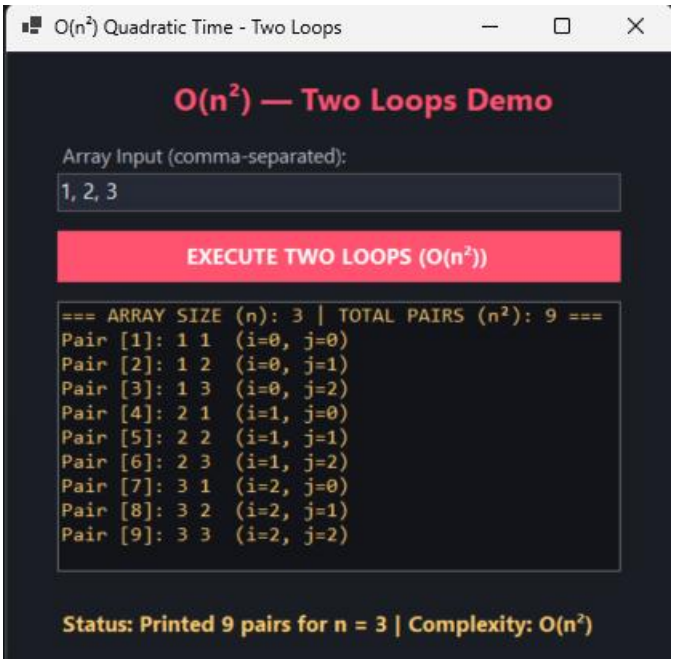
lstTrace.Items.Add($"=== ARRAY SIZE (n): {n} | EXPECTED (n²): {n * n} ===");

// O(n²) Nested Loop Implementation
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        totalIterations++;
```

```
            lstTrace.Items.Add($"[{totalIterations:D2}] Outer (i={i}, val={numbers[i]}) x Inner (j={j}, val={numbers[j]}) => {numbers[i]}{numbers[j]}");
        }
    }

    lblStatus.ForeColor = Color.FromArgb(255, 203, 107);
    lblStatus.Text = $"Status: Executed {totalIterations} iterations for n = {n} | Complexity: O(n²)";
  }
 }
}
```

O(n^2) — Quadratic Time Ex. 2



Code:

```
using System;

using System.Drawing;

using System.Linq;
```

```
using System.Windows.Forms;

namespace O_n____Quadratic_Time_Ex._2
{
  public partial class Form1 : Form
  {
    public Form1()
    {
      InitializeComponent();
    }

    private void btnRun_Click(object sender, EventArgs e)
    {
      lstTrace.Items.Clear();

      string rawArray = txtArrayInput.Text.Trim();

      if (string.IsNullOrEmpty(rawArray))
      {
        lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
        lblStatus.Text = "Status: Please enter array numbers.";
        return;
      }

      int[] numbers;
```

```
try
{
   numbers = rawArray
      .Split(new[] { ',', ' ' }, StringSplitOptions.RemoveEmptyEntries)
      .Select(int.Parse)
      .ToArray();
}
catch (FormatException)
{
   lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
   lblStatus.Text = "Status: Invalid input format. Enter integers only.";
   return;
}

int n = numbers.Length;
int totalIterations = 0;

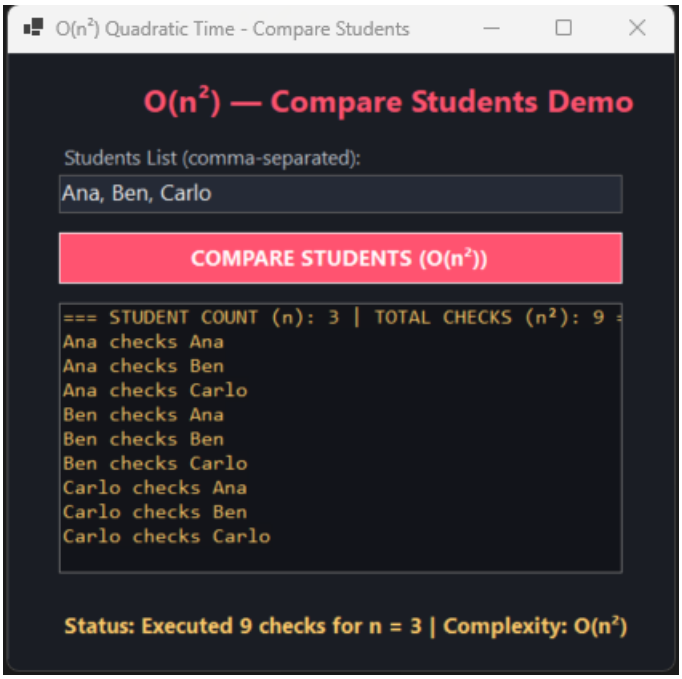
lstTrace.Items.Add($"=== ARRAY SIZE (n): {n} | TOTAL PAIRS (n^2): {n * n} ===");

// Exact O(n^2) Two Loops logic from example slide
for (int i = 0; i < n; i++)
{
   for (int j = 0; j < n; j++)
   {
      totalIterations++;
```

```
            lstTrace.Items.Add($"Pair [{totalIterations}]: {numbers[i]}{numbers[j]}  (i={i}, j={j})");
          }
        }

        lblStatus.ForeColor = Color.FromArgb(255, 203, 107);
        lblStatus.Text = $"Status: Printed {totalIterations} pairs for n = {n} | Complexity: O(n²)";
      }
    }
}
```

O(n^2) — Quadratic Time Ex. 3



Code:

```
using System;

using System.Drawing;

using System.Linq;
```

```
using System.Windows.Forms;

namespace O_n_____Quadratic_Time_Ex._3
{
   public partial class Form1 : Form
{
   private static readonly char[] Separators = new[] { ',', ' ' };

   public Form1()
   {
      InitializeComponent();
   }

   private void BtnRun_Click(object sender, EventArgs e)
   {
      lstTrace.Items.Clear();

      string rawInput = txtStudentsInput.Text.Trim();

      if (string.IsNullOrEmpty(rawInput))
      {
         lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
         lblStatus.Text = "Status: Please enter student names.";
         return;
      }
```

```
    string[] students = rawInput
        .Split(Separators, StringSplitOptions.RemoveEmptyEntries);

    int n = students.Length;
    int totalChecks = 0;

    lstTrace.Items.Add($"=== STUDENT COUNT (n): {n} | TOTAL CHECKS (n²): {n * n} ===");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            totalChecks++;
            lstTrace.Items.Add($"{students[i]} checks {students[j]}");
        }
    }

    lblStatus.ForeColor = Color.FromArgb(255, 203, 107);
    lblStatus.Text = $"Status: Executed {totalChecks} checks for n = {n} | Complexity: O(n²)";
  }
}
}
```

O(2ⁿ) — Exponential Time Ex. 1



Code:

```
using System;

using System.Drawing;

using System.Windows.Forms;


namespace O_2ⁿ____Exponential_Time_Ex._1

{

  public partial class Form1 : Form

  {

    public Form1()

    {

      InitializeComponent();

    }


    private void BtnRun_Click(object sender, EventArgs e)
```

```
    {
        lstTrace.Items.Clear();

        string rawInput = txtSwitchesInput.Text.Trim();

        if (string.IsNullOrEmpty(rawInput) || !int.TryParse(rawInput, out int n) || n < 1)
        {
            lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
            lblStatus.Text = "Status: Enter a valid positive integer (n >= 1).";
            return;
        }

        int combinations = 1;

        lstTrace.Items.Add($"=== INITIAL STATE: 1 combination (0 switches) ===");

        // Exact logic from slide: combinations doubles for each switch added
        for (int i = 0; i < n; i++)
        {
            int prevCombinations = combinations;
            combinations *= 2;
            lstTrace.Items.Add($"Switch #{i + 1}: Doubling ({prevCombinations} * 2) =>
{combinations} combinations");
        }

        lstTrace.Items.Add($"-----------------------------------------------");
```

```
        lstTrace.Items.Add($"Number of combinations: {combinations}");


        lblStatus.ForeColor = Color.FromArgb(0, 225, 255);

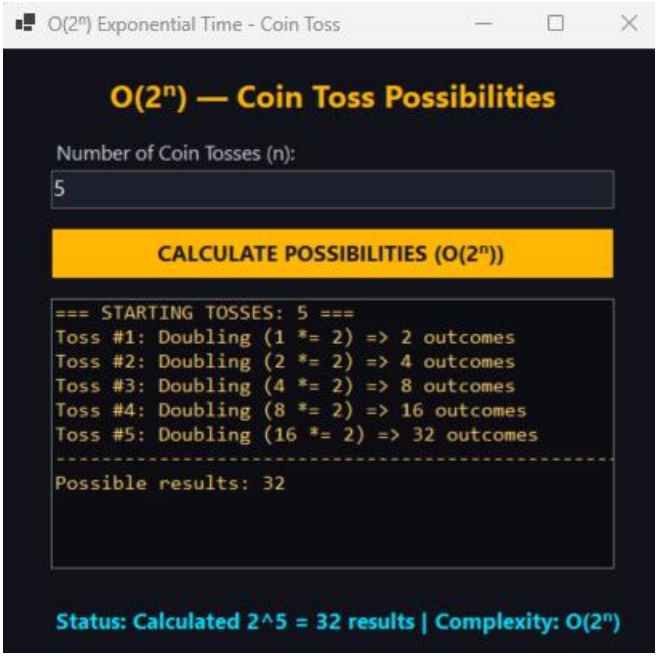
        lblStatus.Text = $"Status: Computed 2^{n} = {combinations} combinations |
Complexity: O(2ⁿ)";

    }

  }

}
```

O(2ⁿ) — Exponential Time Ex. 2



Code:

```
using System;

using System.Drawing;

using System.Windows.Forms;


namespace O_2ⁿ_____Exponential_Time_Ex._2

{
```

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void BtnRun_Click(object sender, EventArgs e)
    {
        lstTrace.Items.Clear();

        string rawInput = txtTossesInput.Text.Trim();

        if (string.IsNullOrEmpty(rawInput) || !int.TryParse(rawInput, out int tosses) ||
tosses < 1)
        {
            lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
            lblStatus.Text = "Status: Enter a valid positive integer (tosses >= 1).";
            return;
        }

        int possibilities = 1;

        lstTrace.Items.Add($"=== STARTING TOSSES: {tosses} ===");

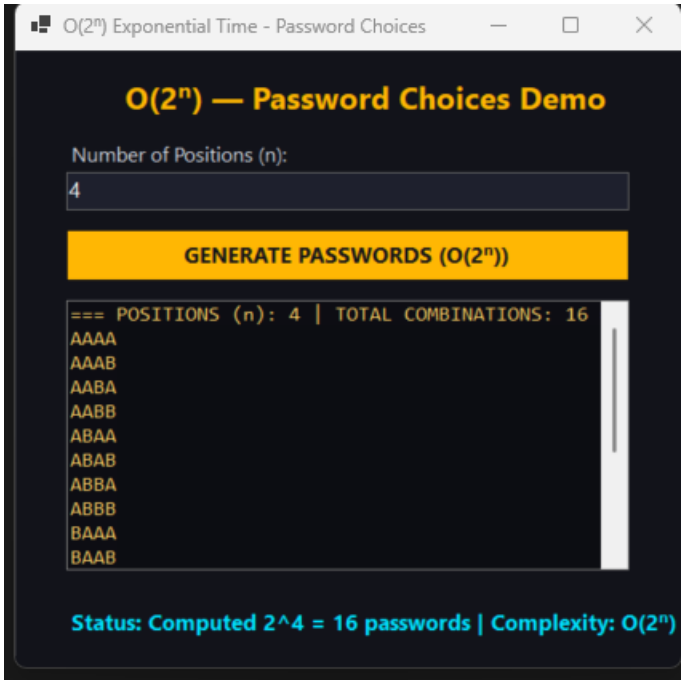
        for (int i = 0; i < tosses; i++)
```

```
        {
            int prevPossibilities = possibilities;
            possibilities *= 2;
            lstTrace.Items.Add($"Toss #{i + 1}: Doubling ({prevPossibilities} *= 2) => {possibilities} outcomes");
        }

        lstTrace.Items.Add($"-----------------------------------------------");
        lstTrace.Items.Add($"Possible results: {possibilities}");

        lblStatus.ForeColor = Color.FromArgb(0, 225, 255);
        lblStatus.Text = $"Status: Calculated 2^{tosses} = {possibilities} results | Complexity: O(2ⁿ)";
    }
  }
}
```

O(2ⁿ) — Exponential Time Ex. 3



Code:

```
using System;

using System.Collections.Generic;

using System.Drawing;

using System.Windows.Forms;


namespace O_2ⁿ____Exponential_Time_Ex._3

{

  public partial class Form1 : Form

  {

    public Form1()

    {

      InitializeComponent();

    }
```

```
    private void BtnRun_Click(object sender, EventArgs e)
    {
        lstTrace.Items.Clear();

        string rawInput = txtPositionsInput.Text.Trim();

        if (string.IsNullOrEmpty(rawInput) || !int.TryParse(rawInput, out int positions) ||
positions < 1 || positions > 10)
        {
            lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
            lblStatus.Text = "Status: Enter a valid position count (1 <= n <= 10).";
            return;
        }

        int possibilities = 1;

        // Compute math calculation from example slide: possibilities *= 2 for each
position
        for (int i = 0; i < positions; i++)
        {
            possibilities *= 2;
        }

        lstTrace.Items.Add($"=== POSITIONS (n): {positions} | TOTAL COMBINATIONS:
{possibilities} ===");

        // Generate binary-style 'A'/'B' string combinations to reflect slide panel
```

```
        List<string> passwords = GeneratePasswordCombinations(positions);

        foreach (string pwd in passwords)
        {
            lstTrace.Items.Add(pwd);
        }

        lstTrace.Items.Add($"-----------------------------------------------");
        lstTrace.Items.Add($"Possible passwords: {possibilities}");

        lblStatus.ForeColor = Color.FromArgb(0, 225, 255);
        lblStatus.Text = $"Status: Computed 2^{positions} = {possibilities} passwords | Complexity: O(2^n)";
    }

    private List<string> GeneratePasswordCombinations(int n)
    {
        List<string> result = new List<string>();
        int total = 1 << n; // 2^n

        for (int i = 0; i < total; i++)
        {
            char[] chars = new char[n];
            for (int bit = 0; bit < n; bit++)
            {
                // Convert binary bit representation into 'A' (0) or 'B' (1)
```

```
            bool isBitSet = ((i >> (n - 1 - bit)) & 1) == 1;
            chars[bit] = isBitSet ? 'B' : 'A';
          }
          result.Add(new string(chars));
        }

        return result;
      }
    }
}
```